

INF264 - Homework 3

Credits: Pierre Gillot, Natacha Galmiche, Odin Hoff Gardå

Instructions

- **Format:** Jupyter notebook
- **Deadline:** Thursday 17th September, 23:59
- **Submission place:** MittUiB

1 Model selection for (polynomial) regression

In this exercise we are going to find the best hyper-parameter settings using k -fold cross-validation. We will use a synthetic dataset consisting of 2 continuous features, x_1 and x_2 , and a single target variable y . The dataset is provided as the file `poly_dataset.csv`. We want to fit a polynomial regression model that can predict y given x_1 and x_2 . In addition, we want to do ridge regression (linear regression with L_2 -regularization) using the polynomial features. In total, we are interested in optimizing two hyper-parameters: the **degree** d of the polynomial (a positive integer), and the **ridge parameter** $\alpha \geq 0$. Solve the task by completing the following steps:

1. Load the dataset `poly_dataset.csv` and store the two features in an array X and the target variable in an array y .
2. Split the dataset into 80% training data and 20% test data. We will not touch the test data until the very end, so for now, pretend it does not exist.
3. Using 5-fold cross-validation, find the best value for d and α :
 - (a) For each pair (d, α) where $d \in \{1, 2, 3, 4, 5, 6\}$ and $\alpha \in \{0, 0.001, 0.01, 0.1\}$:
 - i. Build a polynomial regression model of degree d with ridge parameter α .
 - ii. For each of the 5 cross-validation splits, fit the model on the training part of the data, and compute the MSE over the validation part of the data.
 - iii. Compute and print the mean of the 5 validation MSE scores you obtain for each model (i.e., for each pair (d, α)).
 - (b) Automatically pick the best model (the one with the smallest mean validation MSE).

Tips:

- Use `sklearn.pipeline.Pipeline()` to easily build a model consisting of the following `sklearn` modules:
 - `preprocessing.PolynomialFeatures()` (Generates the polynomial features. Remember to give d as the `degree=` argument),
 - `preprocessing.StandardScaler()` (Standardizes the features), and
 - `linear_model.Ridge()` (Ridge regression model. Remember to give α as the `alpha=` argument).
 - Use `sklearn.model_selection.KFold()` with $k = 5$ to perform 5-fold cross-validation.
 - Use `sklearn.metrics.mean_squared_error()` to compute MSE.
4. Train the best model found by 5-fold cross-validation over the entire training dataset. Then, compute the MSE over the test dataset.

5. Instead of doing k -fold cross-validation, split your training dataset further into a fixed 80% training and a 20% validation dataset. Based on the MSE computed over the fixed validation dataset, pick the best model (the one having the smallest validation MSE). Train this model on the entire training dataset and report the test MSE.
6. How does the test MSE compare for the two methods (k -fold cross-validation versus a fixed validation dataset)?
7. Try to think about the following two questions:
 - Why is standardizing your features important when doing ridge regression?
 - Should we put `StandardScaler()` before or after `PolynomialFeatures()`? What is the difference and does it really matter? (Test it!)

2 Model selection for classification

In this second exercise, we will first illustrate how misleading the accuracy metric can be when assessing a classifier on unbalanced data. Finally, we will cross-validate different classifiers on an unbalanced dataset with a relevant metric (you can get inspiration from the provided template code):

1. Creating your own unbalanced dataset
 - (a) Create 6 randomly generated binary datasets, with the 1st class ratio set to 0.6, 0.75, 0.9, 0.95, 0.98 and 0.99. **Hint:** You can use `make_classification()` from `sklearn.datasets` and set `n_samples=1000`, `n_features=20`, `weights=[ratio]` and `n_classes=2`.
 - (b) For each generated dataset, train and select a K -NN classifier using a regular model selection (no cross validation) and the accuracy as performance metric. **Hint:** Use `accuracy_score()` from `sklearn.metrics`.
 - (c) For each generated dataset, evaluate the selected model on the test dataset using the accuracy score, the F_1 -score, and the balanced accuracy score. In addition, for each generated dataset, plot the confusion matrix. **Hint:** Use `f1_score()`, `balanced_accuracy_score()` and `confusion_matrix()`, from `sklearn.metrics`.
 - (d) Plot the accuracy, balanced accuracy and the F_1 -score with respect to the 1st class ratio in a single figure. Does the accuracy metric appear to assess the quality of your model in an appropriate way?
2. Using a predefined dataset
 - (a) Load the randomly generated dataset from `custom_unbalanced_dataset.pickle`. The dataset contains 10000 rows with a two features and one binary label. **Hint:** Use `np.load()` with `allow_pickle=True` to load the file.
 - (b) Visualize this dataset. How unbalanced is it? **Hint:** You can use `plt.hist()`.
 - (c) Perform model selection on a K -NN classifier. Use k -fold cross-validation with $k = 10$ to find the best value for K . Indicate and justify which metric you decided to use to cross-validate the different models.
 - (d) Train and evaluate the best model with the F_1 -score and the confusion matrix.